CMPSC 463: Design and Analysis of Algorithms (Spring 2026)

Project-1:
Algorithmic Time-Series Segmentation and Condition Analysis Using Water Pump RUL Dataset

By:
Yuliana Gryb

Instructor
Janghoon Yang

Submitted On: 3/15/2026

TABLE OF CONTENTS

1	PROJECT OVERVIEW	3
2	INSTALLATION AND USAGE INSTRUCTIONS	4
3	CODE STRUCTURE EXPLANATION	5
4	ALGORITHM DESCRIPTIONS	6
5	TOY EXAMPLE VERIFICATION	7
6	DATASET DESCRIPTION	8
7	EXECUTION RESULTS	9
8	DISCUSSION AND CONLUSIONS	12

1. PROJECT OVERVIEW

The goal of this project is to analyze time-series sensor data from a water pump machine and relate the sensor behavior to the machine's Remaining Useful Life (RUL). The dataset contains measurements from 52 sensors collected over time. Each row represents a time point and includes sensor readings and the remaining useful life of the machine. Instead of using machine learning libraries, this project focuses on algorithmic techniques.

Three algorithms are used:

- Divide-and-Conquer Segmentation
- Recursive Clustering
- Maximum Subarray (Kadane's Algorithm)

First, the sensor signals are segmented into stable and unstable regions using a recursive divide-and-conquer approach. Next, clustering is applied to group the machine states into four clusters using the sensor measurements. Finally, Kadane's algorithm is used to detect time intervals where the sensor signal changes the most.
The results are compared with machine-health categories created from RUL values. These categories represent different levels of machine health and allow the algorithm results to be interpreted in terms of machine condition.

2. INSTALLATION AND USAGE INSTRUCTIONS

This project is written in Python. The following libraries are required:

- pandas
- numpy
- matplotlib

Install the libraries using:
pip install pandas numpy matplotlib

Place the dataset file inside the data folder.
To run the project, execute the main script:
python main.py

When the script runs it will:

1. Load the dataset and keep the first 10,000 rows.
2. Convert RUL values into four machine-health categories.
3. Run the segmentation algorithm on 10 randomly selected sensors.
4. Save segmentation plots to the results folder.
5. Run the clustering algorithm to create four clusters.
6. Apply Kadane's algorithm to all sensor signals.

The results are printed in the terminal and segmentation plots are saved in:
results/segmentation_plots

3. CODE STRUCTURE EXPLANATION

The code is organized into several files so that each algorithm is implemented separately.

main.py
This file runs the whole project. It loads the dataset, converts RUL values into categories, runs the segmentation algorithm, performs clustering, and runs Kadane's algorithm.

segmentation.py
This file contains the divide-and-conquer segmentation algorithm. It recursively splits a signal when the variance of a segment is too large.

clustering.py
This file contains the recursive clustering algorithm. The algorithm splits data into two groups based on distance from the cluster center until four clusters are created.

kadane.py
This file contains the implementation of Kadane's algorithm, which finds the maximum-sum subarray in a signal.

toy_examples.py
This file contains small example arrays used to verify that the algorithms work correctly before applying them to the real dataset.

4. ALGORITHM DESCRIPTIONS

Divide-and-Conquer Segmentation

The segmentation algorithm divides a sensor signal into segments based on variance.
Steps of the algorithm:

1. Take a portion of the signal.
2. Compute the variance of that segment.
3. If the variance is greater than a threshold, split the segment into two halves.
4. Apply the same process to the two halves.
5. If the variance is small enough, mark the segment as stable.

The recursion stops when the variance is low or the segment becomes small. The total number of segments created is called the Segmentation Complexity Score.

Recursive Clustering

Clustering groups similar machine states together.
The algorithm works as follows:

1. Start with all data points in one cluster.
2. Compute the center of the cluster.
3. Measure the distance of each point from the center.
4. Split the data into two groups based on the median distance.
5. Repeat the splitting process until four clusters are produced.

Each cluster represents a group of time instances with similar sensor measurements.

Kadane's Algorithm

Kadane's algorithm is used to find the time interval where a sensor signal changes the most.
First, the signal is preprocessed:
$d[i] = |sensor[i] - sensor[i-1]|$
Then the average value of d is subtracted:
$x[i] = d[i] - mean(d)$

Kadane's algorithm then finds the continuous subarray with the largest sum in x. This interval represents the period with the strongest sustained signal deviation.

5. TOY EXAMPLE VERIFICATION

Kadane's algorithm was verified using the array [1, -2, 3, 4, -1, 2, -5]. The algorithm correctly identified the maximum subarray [3, 4, -1, 2] with a total sum of 8, returning the indices (2, 5).

The segmentation algorithm was tested on the signal [1,1,1,10,10,10,1,1,1]. Using a variance threshold of 2, the algorithm produced a single segment (0,9), meaning the entire signal was treated as one stable segment.

```
(2, 5, 8)
[(0, 9)]
```

These toy examples confirm that the implementations behave correctly before applying them to the full dataset.

6. DATASET DESCRIPTION

The dataset used in this project is the Water Pump RUL Predictive Maintenance dataset. Each row represents a time point and contains:

- a timestamp
- 52 sensor measurements (sensor_00 to sensor_51)
- the Remaining Useful Life (rul) in hours

Only the first 10,000 rows of the dataset were used in this project.

RUL to 4-Category Transformation

The RUL values were converted into four machine-health categories using quantiles. Three thresholds were calculated:

- Q10 – 10th percentile of RUL
- Q50 – 50th percentile of RUL
- Q90 – 90th percentile of RUL

The categories were defined as:
Extremely Low RUL
$rul < Q10$
Moderately Low RUL
$Q10 \leq rul < Q50$
Moderately High RUL
$Q50 \leq rul < Q90$
Extremely High RUL
$rul \geq Q90$
These categories allow the algorithm results to be compared with machine health levels.

7. EXECUTION RESULTS

Task 1 – Segmentation

Ten sensors were randomly selected and segmented using the divide-and-conquer algorithm.
Segmentation plots were created for each sensor and saved in the results folder.

```
Segmentation Complexity Scores
sensor_13 : 8
sensor_26 : 34
sensor_44 : 107
sensor_02 : 7
sensor_22 : 12
sensor_10 : 224
sensor_11 : 10
sensor_37 : 53
sensor_47 : 81
sensor_18 : 255
```

10 segmentation plots were generated and saved in ***results/segmentation_plots.***
Sensors with higher segmentation scores show more changes in the signal.

Task 2 – Clustering

The clustering algorithm was applied to all 10,000 time instances using the 52 sensor values.

The algorithm produced four clusters. For each cluster, the number of samples from each RUL category was counted.

```
Cluster Majority Counts

Cluster 0
rul_class
1    2001
0     253
2     246
Name: count, dtype: int64

Cluster 1
rul_class
2    1305
1     770
0     242
3     183
Name: count, dtype: int64

Cluster 2
rul_class
2    1642
1     807
0      51
Name: count, dtype: int64

Cluster 3
rul_class
3    817
2    807
0    454
1    422
Name: count, dtype: int64
```

This shows that the clusters correspond to different machine-health states.

Task 3 – Maximum Subarray

Kadane's algorithm was applied to each sensor signal.
For each sensor, the time interval with the strongest deviation was identified. The RUL categories within that interval were examined to see which machine-health state occurred most often.

```
Kadane Majority Class Per Sensor
sensor_00 -> class 2
sensor_01 -> class 2
sensor_02 -> class 0
sensor_03 -> class 1
sensor_04 -> class 2
sensor_05 -> class 2
sensor_06 -> class 2
sensor_07 -> class 1
sensor_08 -> class 1
sensor_09 -> class 1
sensor_10 -> class 1
sensor_11 -> class 2
sensor_12 -> class 2
sensor_13 -> class 1
sensor_14 -> class 0
sensor_16 -> class 2
sensor_17 -> class 1
sensor_18 -> class 1
sensor_19 -> class 0
sensor_20 -> class 2
sensor_21 -> class 2
sensor_22 -> class 0
sensor_23 -> class 2
sensor_24 -> class 0
sensor_25 -> class 1
sensor_26 -> class 2
sensor_27 -> class 3
sensor_28 -> class 3
sensor_29 -> class 1
```

```
sensor_30 -> class 2
sensor_31 -> class 2
sensor_32 -> class 1
sensor_33 -> class 2
sensor_34 -> class 2
sensor_35 -> class 1
sensor_36 -> class 1
sensor_37 -> class 0
sensor_38 -> class 1
sensor_39 -> class 1
sensor_40 -> class 1
sensor_41 -> class 1
sensor_42 -> class 1
sensor_43 -> class 1
sensor_44 -> class 2
sensor_45 -> class 2
sensor_46 -> class 1
sensor_47 -> class 1
sensor_48 -> class 1
sensor_49 -> class 2
sensor_51 -> class 1
```

Sensors whose strongest deviations occur in low RUL categories may indicate early machine degradation.

8. DISCUSSION AND CONLUSIONS

The results show that algorithmic techniques can be used to analyze machine sensor data and identify patterns related to machine health.
The segmentation algorithm identified differences in signal stability across sensors. Some sensors showed very few segments, indicating stable behavior, while others showed many segments, indicating frequent changes in the signal.

The clustering algorithm grouped machine states into four clusters that roughly correspond to the RUL categories. This suggests that sensor measurements contain information about machine health. Kadane's algorithm identified intervals where sensor signals changed significantly. These intervals sometimes occurred when the machine had low remaining useful life, suggesting that certain sensors may help detect early signs of machine failure.

Challenges in this Project
One challenge was choosing a good variance threshold for segmentation. If the threshold is too small, the algorithm creates too many segments. If the threshold is too large, important changes in the signal may not be detected.
Another challenge was interpreting clustering results, since clustering does not directly use the RUL categories.

Limitations and Possible Improvements
This project focuses on classical algorithms and does not use machine learning models. While this highlights algorithm design, more advanced methods could potentially produce better predictions.
Possible improvements include:

- testing different segmentation thresholds
- combining multiple sensor signals into new features
- using more advanced clustering methods
- comparing results with machine learning approaches